Part 2 Report: Reproducible Traffic Analytics Pipeline

Prepared by: Kyi Htoo Aung | Smart City Traffic Intelligence Capstone, Part 2 | Python (pandas, NumPy, Matplotlib, logging, argparse)

1. Objective and design

The goal was to turn the Part 1 analysis into a reliable, repeatable workflow. **pipeline.py** is the single entry point: it configures logging, loads and validates the raw CSV, runs twelve separately logged cleaning steps, then calls **feature_engineering.py** and **visualizations.py**. A command-line application, **cli_app/traffic_cli.py**, answers questions from the processed table. One command (*python pipeline.py*) rebuilds every output, and every failure path ends with a logged ERROR and a non-zero exit code instead of an unhandled crash.

2. Methodology: loading, validation and cleaning

The CSV is read inside *try/except* blocks for missing, empty, undecodable and unparseable files (no bare *except*); the text "None" in *holiday* is preserved as a value. The schema is validated before any processing. Each cleaning step then logs its own result:

Step	What was found and done	Rows affected
Standardise categories	Inconsistent case, e.g. "Sky is Clear" vs "sky is clear", "SQUALLS" vs "squalls"; lower-cased	1,730 modified
Parse date/times	Explicit format; invalid timestamps would be dropped	0 invalid
Exact duplicates	Identical rows removed	17 dropped
Duplicate timestamps	Same hour recorded once per weather description; volume identical, so one row per hour kept (most severe weather)	7,612 dropped
Impossible temperatures	Outside 220-330 K (0 K is absolute zero): set to missing	10 flagged
Impossible rainfall	Above 305 mm/h (world one-hour record; reading was 9,831.3 mm): set to missing	1 flagged
Monthly median imputation	Loop over each year-month; fill with that month's median (Jan/Feb 2014, about -13 °C, not the +8 °C global median)	11 imputed
Sensor anomalies	Busy-period hours under 5% of the typical volume for that hour and weekday (e.g. 23-24 Jul 2016); kept and flagged	32 flagged
Final checks	No missing values, one row per hour, sorted	40,575 rows remain

3. Feature engineering

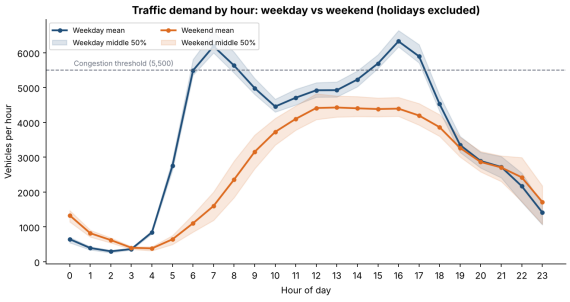
The table grows from 40,575 x 14 to 40,575 x 54 (both shapes logged at INFO). **Time:** hour, day of week, weekend, holiday (the midnight-only label is spread to the whole date), weekday peak hours from Part 1, and sin/cos encodings for hour, weekday and month. **Weather:** Celsius temperature, 11 one-hot weather columns, precipitation, low-visibility (Mist, Fog, Haze, Smoke) and severe-weather (Thunderstorm, Squall, Snow) indicators, plus a 0-10 severity score. **Scaled:** temperature z-score, cloud cover min-max and log-scaled rain. **Target:** *congestion_category* splits hours at the volume quartiles (Q1 = 1,254.5, median = 3,431, Q3 = 4,953, logged at DEBUG) into four equal classes, Low, Medium, High and Severe; *is_congested* keeps the Part 1 rule (> 5,500) for comparability.

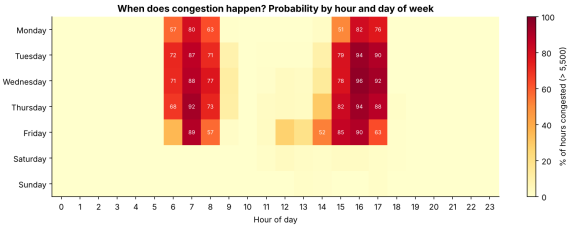
4. Logging

Every module uses *logging.getLogger(__name__)*; handlers are configured only in the two entry points. Records go to the console and to *pipeline.log* with the format *timestamp* | *level* | *module* | *message*. INFO marks milestones (file loaded with rows x columns, shapes, each figure saved with its path); WARNING records every dropped, modified, imputed or flagged row with the count and reason; ERROR (with *exc_info=True*) records anything that stops a run; DEBUG holds intermediate values (quartiles, monthly medians, scaling parameters) and appears only with *--debug*. *print()* is used solely for the CLI's answers.

5. Findings from the visualisations

Demand by hour (fig01, right): weekdays have two commuter peaks, about 6,180 vehicles at 07:00 and 6,330 at 16:00, both above 5,500; weekends have one midday plateau of about 4,430. **Congestion heatmap (fig05, right):** congestion is confined to weekdays 06-08 and 15-17, reaching 94-96% of hours on Tuesday to Thursday at 16:00, and is almost absent at weekends. **Distribution (fig02):** volumes are bimodal (quiet nights below 1,000, busy days of 4,000-6,500), so averages hide the real pattern and quartile classes are more informative.





Temperature (fig03): a weak link (r = 0.14); averages are flat between -20 °C and 20 °C and rise above 25 °C only because warm hours are summer afternoons. **Weather (fig04):** among well-sampled conditions averages range from Haze (3,677) to Fog (2,654), yet Fog has the highest congestion share (20%) because it forms during the morning rush. **Over time (fig06):** traffic dips in December and January, peaks in late summer, and the yearly average rises from about 3,200-3,300 (2012-2016) to 3,377 in 2017, around the shaded 2014-2015 data gap.

6. Mini application

The CLI offers six commands: *summary*, *query* (traffic and weather for a date/hour, with the nearest hours shown if it falls in a data gap), *peak*, *compare* (weekdays carry 91% more morning traffic than weekends in 2017), *recommend* (least congested hours on a chosen day, e.g. Friday 10:00 or after 19:00, avoiding 16:00) and *weather*. Each invocation and its arguments are logged at INFO; malformed dates, unknown days or conditions and out-of-range values produce a clear ERROR and exit code 2 rather than a traceback.

7. Conclusions and next steps

- Congestion is a **weekday commuting problem** (07:00-08:00 and 15:00-17:00); time of day explains far more than weather or temperature, confirming Part 1 with cleaner, de-duplicated data.
- Data quality issues are real and systematic (inconsistent labels, duplicated hours, impossible readings, sensor outages); the logged pipeline makes every correction auditable and repeatable.
- **Limitations:** one sensor location, a 10-month gap, a heuristic anomaly rule and one weather label per hour. The feature table (with cyclical, weather-risk and quartile target features) is the direct input to the Part 3 models.